

Problem set — Module 1 (Chapter 1)

Work these with pencil before reading solutions. The single most important exercise is **P3**: reproduce the quicksort recurrence solve from scratch. If you can do that unaided, you've got the module.

Solutions are at the bottom. Book exercise numbers noted where relevant.

Problems

P1 (warm-up — reframing Big-O). For the loop below, give the *exact* number of times the comparison `j < n` is evaluated, as a function of n . Then state it in Big-O. Notice how much the exact count says that O throws away.

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        // constant work
```

P2 (harmonic numbers). (a) Compute H_1, H_2, H_3, H_4 as exact fractions. (b) Using $H_N \approx \ln N + \gamma$ with $\gamma = 0.5772$, estimate H_{1000} , and compare to the true value 7.4855. (c) Explain in one sentence why H_N grows without bound even though the terms $\frac{1}{k} \rightarrow 0$.

P3 (the main event — solve the quicksort recurrence). Starting from

$$C_N = (N + 1) + \frac{1}{N} \sum_{1 \leq j \leq N} (C_{j-1} + C_{N-j}), \quad C_0 = C_1 = 0,$$

derive $C_N = 2(N + 1) \left(H_{N+1} - \frac{3}{2} \right)$ with no peeking at the notes. Name each technique as you use it (symmetry, multiply-and-subtract, telescoping).

P4 (best & worst case). (a) What input makes this quicksort do the *most* compares? Give the worst-case compare count exactly (leading term is enough), and show it's $\sim N^2/2$. (b) Contrast with the average $2N \ln N$. Roughly how many times faster is the average case than the worst case at $N = 1,000,000$?

P5 (Book Ex. 1.14 — a sibling recurrence). Solve

$$A_N = 1 + \frac{2}{N} \sum_{1 \leq j \leq N} A_{j-1}, \quad \text{for } N > 0, A_0 = 0.$$

(This is the same machinery as P3 but cleaner; it's the shape that counts "stages"/nodes.)

P6 (Book Ex. 1.16 — a taste of tuning). On average, how many subarrays of size 2 or less are encountered when quicksorting a random file of size N ? (Hint: think about, for a fixed element, the probability it appears as the pivot of a tiny subarray — or reason about the recursion-tree size. A clean heuristic estimate is enough.)

P7 (conceptual — why we count compares, not seconds). The book models quicksort's running time as $4C + 11B + 35A$ on one machine. Suppose a new machine makes compares $3\times$ cheaper but exchanges $2\times$ more expensive. Explain what you'd have to recompute, and what you *wouldn't*. What's the general lesson about separating the algorithm from the machine?

Solutions

P1. Outer loop runs $i = 0, \dots, n - 1$. For each i , the inner test $j < n$ is evaluated once per inner iteration *plus one final failing test*: it's true n times ($j = 0, \dots, n - 1$) then false once, so $n + 1$ evaluations per outer pass. Total = $n(n + 1) = n^2 + n$. In Big-O: $O(n^2)$. Notice $O(n^2)$ hides the $+n$ and the leading constant 1 — for comparing two $O(n^2)$ algorithms, that constant is exactly what matters. Exact counts are the currency of this course.

P2. (a) $H_1 = 1$; $H_2 = \frac{3}{2}$; $H_3 = \frac{11}{6}$; $H_4 = \frac{25}{12} \approx 2.0833$. (b) $\ln 1000 = 6.9078$; $+ 0.5772 = 7.4850$. True value 7.4855 — off by 0.0005, about 0.007%. (The approximation improves as N grows; the next correction term is $+\frac{1}{2N}$.) (c) Because the *partial sums* diverge: grouping terms shows $H_N \geq 1 + (\text{halves that keepadding} \geq \frac{1}{2})$, so the sum crawls to infinity like $\ln N$ even though individual terms vanish.

P3. (Full derivation — this mirrors §3 of the notes.)

- **Symmetry:** in $\sum (C_{j-1} + C_{N-j})$, as $j = 1, \dots, N$ both pieces range over $C_0, \dots, C_{N-1}$, so the sum is $2 \sum_{1 \leq j \leq N} C_{j-1}$. Hence $C_N = N + 1 + \frac{2}{N} \sum_{1 \leq j \leq N} C_{j-1}$.
- **Clear the fraction:** $NC_N = N(N + 1) + 2 \sum_{1 \leq j \leq N} C_{j-1}$.
- **Multiply-and-subtract:** write the same for $N - 1$: $(N - 1)C_{N-1} = (N - 1)N + 2 \sum_{1 \leq j \leq N-1} C_{j-1}$. Subtract: $NC_N - (N - 1)C_{N-1} = [N(N + 1) - (N - 1)N] + 2C_{N-1} = 2N + 2C_{N-1}$.
- Rearrange: $NC_N = (N + 1)C_{N-1} + 2N$.
- **Telescope:** divide by $N(N + 1)$: $\frac{C_N}{N+1} = \frac{C_{N-1}}{N} + \frac{2}{N+1}$. With $T_N = C_N/(N + 1)$, $T_N = T_{N-1} + \frac{2}{N+1}$, $T_1 = 0$. Iterate: $T_N = 2 \sum_{3 \leq k \leq N+1} \frac{1}{k} = 2(H_{N+1} - \frac{3}{2})$.
- Multiply back: $C_N = 2(N + 1)(H_{N+1} - \frac{3}{2})$. ✓ (Using $H_{N+1} \approx \ln N$ gives $\approx 2N \ln N - 1.846N$.)

P4. (a) Worst case: an already-sorted (or reverse-sorted) array. Every partition puts the pivot at an end, so the subarray sizes are $N - 1$ and 0 . Compares: $(N + 1) + N + (N - 1) + \dots \approx \sum_k k \approx N^2/2$. (Ironically the "easiest-looking" input.) (b) Ratio $\approx \frac{N^2/2}{2N \ln N} = \frac{N}{4 \ln N}$. At $N = 10^6$: $\ln(10^6) \approx 13.8$, so ratio $\approx \frac{10^6}{55.2} \approx 18,000$ — about **18,000× faster** on average than worst case. This gap is why randomization / median-of-three (which make the worst case astronomically unlikely) matter.

P5. Same steps as P3.

- $N A_N = N + 2 \sum_{1 \leq j \leq N} A_{j-1}$. Subtract the $N - 1$ version:
 $N A_N - (N - 1) A_{N-1} = [N - (N - 1)] + 2 A_{N-1} = 1 + 2 A_{N-1}$.
- So $N A_N = (N + 1) A_{N-1} + 1$. Divide by $N(N + 1)$: $\frac{A_N}{N+1} = \frac{A_{N-1}}{N} + \frac{1}{N(N+1)}$.
- Use $\frac{1}{N(N+1)} = \frac{1}{N} - \frac{1}{N+1}$ (partial fractions). The sum of $(\frac{1}{k} - \frac{1}{k+1})$ telescopes, so the growth is $A_N \sim 2 \ln N$. (The exact closed form is fiddly; the takeaway is that the *method* is identical to P3, and this is a good example of "the elementary trick works but is clumsy — recurrences deserve a real theory," which is Module 2.)

P6. Heuristic: the recursion tree for quicksort on N elements has about N internal partitioning nodes, and the subproblems shrink geometrically-ish. Subarrays of size ≤ 2 are the small leaves of that tree. A clean estimate: on average roughly $N/3$ subarrays of size ≤ 2 are encountered (each element is the pivot of exactly one stage, and a constant fraction of stages act on tiny subarrays). This is exactly the kind of quantity you'd want to nail down *precisely* to choose the cutoff M for switching to insertion sort — which is why the book pushes toward better recurrence tools. (Exact treatment: Modules 2-3.)

P7. You'd **recompute the machine coefficients** (the $4 \rightarrow \sim 1.33$ for compares, the $11 \rightarrow 22$ for exchanges) and re-form the linear combination. You would **not** recompute A, B, C — those are frequencies determined by the algorithm and the input distribution, totally independent of the machine. Lesson: analyze the algorithm-dependent quantities once, mathematically; fold in machine constants only at the very end. That factoring is what makes a 50-year-old analysis still predict today's hardware.

Stretch / from the booksite

If you want more, the booksite aofa.cs.princeton.edu has the full Chapter 1 exercise set with some solutions. Book Ex. 1.17-1.19 (adding an insertion-sort cutoff M and *optimizing* M) are a great preview of why precise analysis pays off — but

they need Module 2/3 tools to finish cleanly, so save them as a callback once we've done recurrences.